Report SPM Project

Simone Stanganini

Contents

Introduction

For this project i choose the version 1 of the Distributed Wavefront computation.

Project Structure:

- **spm_ff** : the folder for the fastflow version .
- **spm_omp** : the folder that contains the openMPI version of the code.
- **Makefile** : all the command for compile and execute the programs.
- **scripts** : all the scripts for running the code in the smpcluster.
- **results** : files that contain the result of the execution in the cluster machines.

Commands

```
1. Make
```

This command will compile both versions of the code, and the executables, ff for the fastflow version and omp for the MPI version, will be placed in the home directory of the project.

```
1. Make test_ff
```

This test will go through various execution of the fastflow version on the current machine that the command is executed. It will compute 3 matrices of size 128, 1024 and 4096, with both 8 and 16 threads.

```
1. Make test_omp
```

This test will spawn 4 processes in the current machine, and it will compute 4 matrices of size 128, 512, 1024 and 4096.

```
1. Make run_ff
```

This command will execute the script scripts/script_ff.sh on the spmcluster, with sbatch so it will go in the jobs queue, and it will take 1 node and do varius tests of the fastflow version.

The results will be printed in the file results/out_omp.log.

```
1. Make run_omp
```

This command will execute the script script_omp.sh on the spmcluster, with sbatch, this script will take 4 nodes for the computation.

Parallelization Strategy and Implementation

Fast Flow Version

For this version, given the type of computation to be performed, i chose to create a farm with a single emitter that also functions as a collector, which was obviously removed. In this case, i decided to compute each diagonal of the matrix without ever stopping, so i tried to maximize the use of threads for each diagonal without ever blocking their execution. Once a diagonal was completed, for ease of implementation, i would wait for the threads to finish executing, and upon their completion, i would proceed with the computation of a new diagonal.

The communication between the worker and the emitter is on-demand, so I developed an algorithm that initially sends tasks to all the workers, which are simply the indices of the shared matrix to be computed by all the threads. Once the computation is done, they send back a message via their feedback_channel. At this point, the emitter, upon seeing that a worker has finished its computation, immediately sends a new task to that worker if the diagonal is not yet complete.

It’s important to note that the individual workers write to the shared matrix without the need for synchronization, as the synchronization is inherent in the way the emitter distributes the tasks to the workers. Once all tasks are completed, the emitter broadcasts an EOS to shut down all the workers.

MPI Version

In this version of the code, i used a solution very similar to the fast_flow version, where one node handles the task management and the matrix management(what I will also call the emitter in this case) while the other nodes handle the actual computation of the matrix, which are again the workers.

Since we are dealing with processes, i had to keep a copy of the matrix on each individual node, and this required me to broadcast every time a diagonal was completed by the emitter node to the workers. During the matrix computation, the emitter sends tasks to the workers in the form of a message containing the indices to compute. Once a worker completes the computation on its local matrix, it doesn't write the result but sends it to the emitter node, which stores it in the original matrix and then broadcasts it to all other workers once the current diagonal is finished.

For implementation reasons, every time tasks are sent to as many workers as there are available, we stop and wait for the results. Once all results from the workers have been received, we continue with new tasks, repeating this process until the matrix is fully computed.

All calls made to the openMPI library are non-blocking, specifically using MPI_Isend and MPI_Irecv, so each time we wait for results, the program uses MPI_Wait to pause execution. For the broadcast, as per the worker logic, the emitter performs an MPI_Ibcast and then waits until the message reaches all the workers.

One issue I encountered was that in 8 to 10 executions, the MPI_Wait function, which waits for the reception of the broadcast sent with MPI_Ibcast, didn’t work properly. As a result, the emitter resumed execution and sent new tasks while one worker had not yet received the broadcast message, causing the two calls to overlap and leading to incorrect execution. To address this, i used the MPI_Barrier function after the broadcast to mitigate this bug.

Additionally, it's worth noting that the individual workers use openMP internally for computation, parallelizing the for loop used for calculating the diagonal index. This parallelization causes a different order of operations in various code executions depending on the threads used by openMP. This results in a slight loss of precision during the addition and multiplication of doubles, so while the printed results appear identical, comparing them with the sequential version of the code shows that they are not exactly the same down to the bit. However, I considered this acceptable, as it’s an unsolvable issue and I preferred to keep a parallelized version even within the individual nodes.

Tests Results

Below are the tables used for performance calculations and the results of the various executions of the three versions of the code. Note that all times are expressed in milliseconds (ms):

- Fast Flow :

	8	16	32	64	128	256	512	724	1024	1448	2048	2896	4096	5792
2	0.478	0.477	0.73	0.161	4.515	16.506	93.489	176.688	498.776	937.025	4698.683	7978.358	73711.21	101845.9
4	0.745	1.315	0.837	0.177	3.432	9.439	48.645	95.088	252.595	520.09	2287.985	5003.655	36324.97	68246.33
6	1.416	0.83	1.012	0.145	2.547	7.142	34.413	69.02	173.01	382.509	1599.758	3794.075	24754.93	53281.33
8	1.763	1.644	1.689	0.171	2.608	6.854	29.29	62.35	142.882	330.632	1307.055	3253.673	30545.15	86863.05
10	1.323	1.471	1.23	0.145	3.272	6.587	25.042	52.649	118.065	273.008	1051.883	2263.124	19908.33	39345.93
12	2.77	1.611	2.157	0.144	3.5	6.685	22.474	46.948	103.514	237.348	888.541	2074.384	15448.02	38566.97
14	2.305	2.276	2.222	0.17	3.185	6.767	22.885	45.543	95.131	215.031	774.604	1784.486	13456.08	32828.23
16	2.979	2.374	2.559	0.158	4.776	10.647	38.258	76.23	145.484	308.035	837.03	1952.958	11038.83	32165.94
20	3.38	3.047	2.626	0.145	5.581	11.263	38.794	75.193	144.87	303.539	790.056	1692.541	10308.11	26778.07
24	3.057	3.684	3.231	0.171	5.731	11.707	39.637	76.137	144.744	300.314	770.615	1655.819	9058.64	25668.73
28	6.994	3.841	3.978	0.144	6.218	12.237	41.092	79.38	148.896	307.496	781.042	1722.764	8608.825	23511.51
32	23.061	7.467	18.203	841.435	2478.951	5572.575	12397.97	17922.28	26809.12	38539.65	54408.44	77960.35	110310.6	152612.9

- MPI :

	8	16	32	64	128	256	512	724	1024	2048	2896	4096	5792	8192
4	22.891	15.644	28.931	35.013	75.612	211.532	771.333	1434.969	2832.489	12583.04	25879	83049.73	158738.1	624580.4
6	25.95	28.889	34.324	0.051019	86.093	210.486	657.167	1151.68	2082.649	2082.649	16290.82	51130.16	111404.8	384995.5
8	29.947	31.951	39.546	54.728	107.458	259.702	705.12	1194.265	2109.526	7072.572	13962.24	38410.77	92795.02	275224

- Sequential :

8	16	32	128	256	512	724	1024	1448	2048	2896	4096	5792
0.011	0.015	0.034	0.895	4.912	64.479	101.542	479.701	782.887	7450.455	9335.961	142845.1	146029.7

Performance Analysis

Strong Scalability

- **Fast Flow :**

Below is the study on speed-up for a 1024*1024 matrix, considering that the execution time of the sequential version of the program is approximately 479 ms.

Threads	**Time (ms)**	**Speedup (S) =** $\frac{T_{seq}}{T_{par}}$	**Efficiency (E) =** $\frac{S}{p}$
2	498	$\frac{479}{498} \approx 0.96$	$\frac{0.96}{2} = 0.48$
4	252	$\frac{479}{252} \approx 1.90$	$\frac{1.90}{4} = 0.47$
8	142	$\frac{479}{142} \approx 3.37$	$\frac{3.37}{8} = 0.421$
16	145	$\frac{479}{145} \approx 3.30$	$\frac{3.30}{16} = 0.206$
24	144	$\frac{479}{144} \approx 3.32$	$\frac{3.32}{24} = 0.138$

Below is the same study on a 4096*4096 matrix, with the execution time of the sequential algorithm being approximately 142845 ms..

Threads	**Time (ms)**	**Speedup (S) =** $\frac{T_{seq}}{T_{par}}$	**Efficiency (E) =** $\frac{S}{p}$
2	73711	$\frac{142845}{73711} \approx 1.93$	$\frac{1.93}{2} = 0.965$
4	36324	$\frac{142845}{36324} \approx 3.93$	$\frac{3.93}{4} = 0.982$
8	30545	$\frac{142845}{30545} \approx 4.67$	$\frac{4.67}{8} = 0.583$
16	11038	$\frac{142845}{11038} \approx 12.94$	$\frac{12.94}{16} = 0.808$

24	9058	$\frac{142845}{9058} \approx 15.77$	$\frac{15.77}{24} = 0.657$

- **MPI :**

Below is the study on a 1024*1024 matrix:

Processors	**Time (ms)**	**Speedup (S) =** $\frac{T_{seq}}{T_{par}}$	**Efficiency (E) =** $\frac{S}{p}$
4	2832	$\frac{479}{2832} \approx 0.169$	$\frac{0.169}{4} = 0.0422$
6	2082	$\frac{479}{2082} \approx 0.23$	$\frac{0.23}{6} = 0.0383$
8	2109	$\frac{479}{2109} \approx 0.22$	$\frac{0.22}{8} = 0.0275$

Below is the study on a 4096*4096 matrix:

Processors	**Time (ms)**	**Speedup (S) =** $\frac{T_{seq}}{T_{par}}$	**Efficiency (E) =** $\frac{S}{p}$
4	83049	$\frac{142845}{83049} \approx 1.72$	$\frac{1.72}{4} = 0.43$
6	51130	$\frac{142845}{51130} \approx 2.79$	$\frac{2.79}{6} = 0.465$
8	38410	$\frac{142845}{38410} \approx 3.71$	$\frac{3.71}{8} = 0.463$

Weak Scalability

To conduct the weak scalability study, I doubled both the number of processors and the size of the input elements.

- **Fast Flow :**

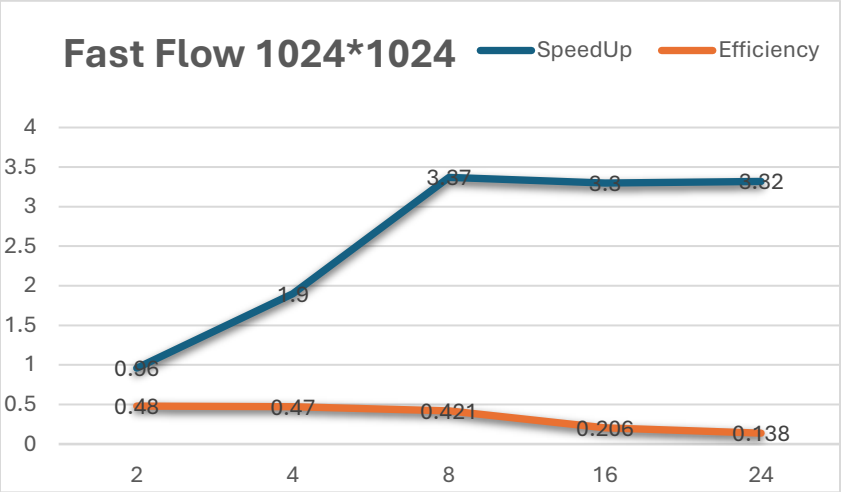
Threads	Matrix	Time (ms)	Speedup (S) = $\frac{T_{seq}}{T_{par}}$	Efficiency (E) = $\frac{S}{p}$
2	512	93	$\frac{64}{93} \approx 0.68$	$\frac{0.68}{2} = 0.34$
4	724	95	$\frac{101}{95} \approx 1.06$	$\frac{1.06}{4} = 0.265$
8	1024	142	$\frac{479}{142} \approx 3.37$	$\frac{3.37}{8} = 0.421$
16	1448	308	$\frac{782}{308} \approx 2.53$	$\frac{2.53}{16} = 0.158$

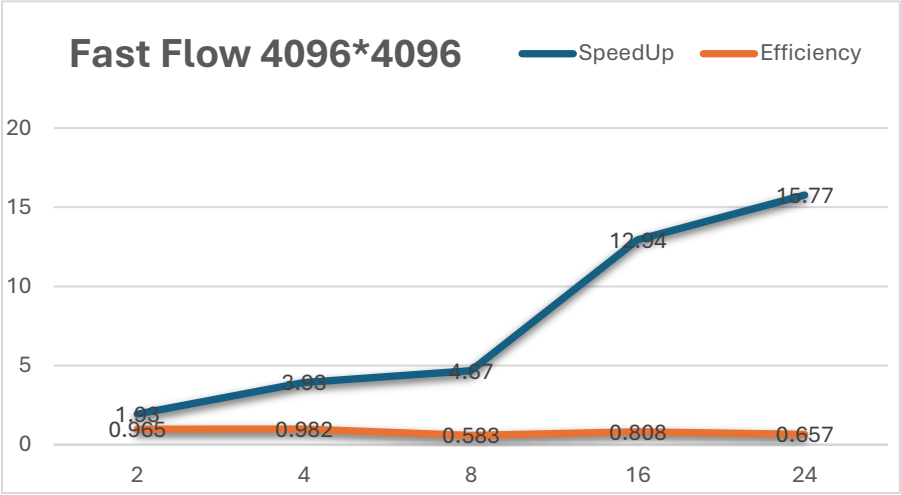
- **MPI :**

Processors	Matrix	Time (ms)	Speedup (S) = $\frac{T_{seq}}{T_{par}}$	Efficiency (E) = $\frac{S}{p}$
2	512	771	$\frac{64}{771} \approx 0.08$	$\frac{0.08}{2} = 0.04$
4	724	1151	$\frac{101}{1151} \approx 0.087$	$\frac{0.087}{4} = 0.021$
8	1024	2109	$\frac{479}{2109} \approx 0.22$	$\frac{0.22}{8} = 0.0275$

Plots and Conclusions

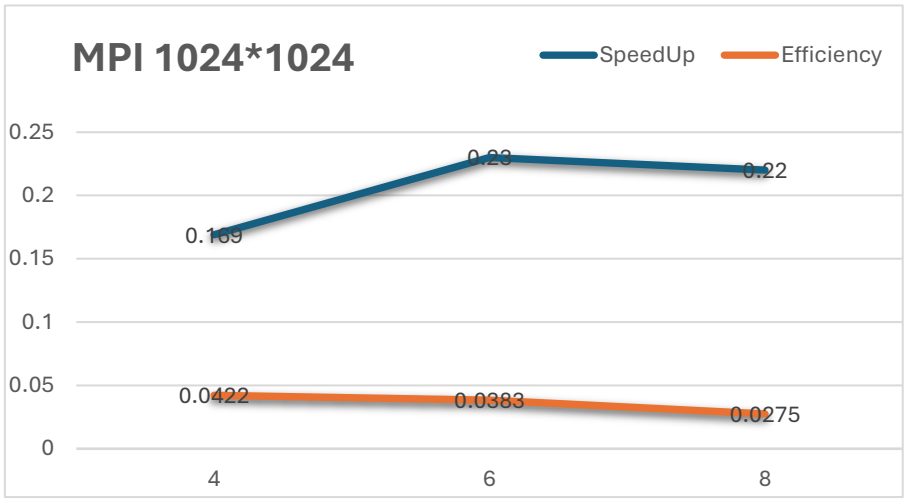
- Strong Scaling

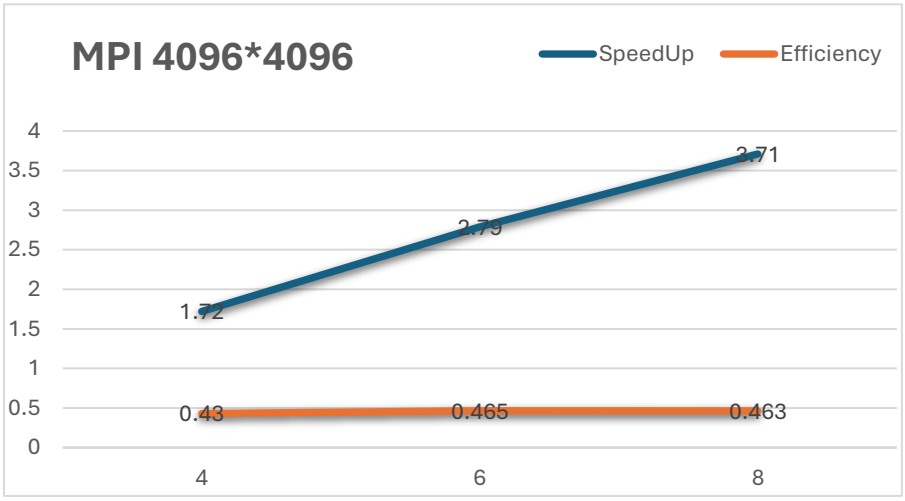




The improvement in speed-up with an increasing number of threads is noticeable up to a certain point. Efficiency drops rapidly beyond 8 threads, indicating that there is an overhead associated with managing many threads relative to the fixed problem size (1024x1024).

In contrast, for the 4096 matrix, the performance is significantly better. The speed-up rises to approximately 15.77 with 24 threads, and efficiency remains high up to 16 threads, indicating that the code scales well with larger problems.

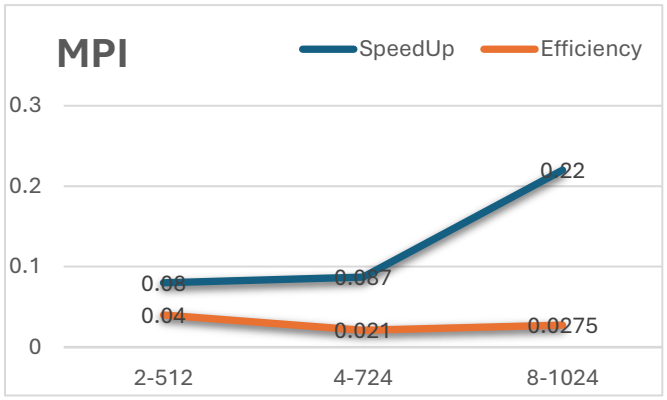


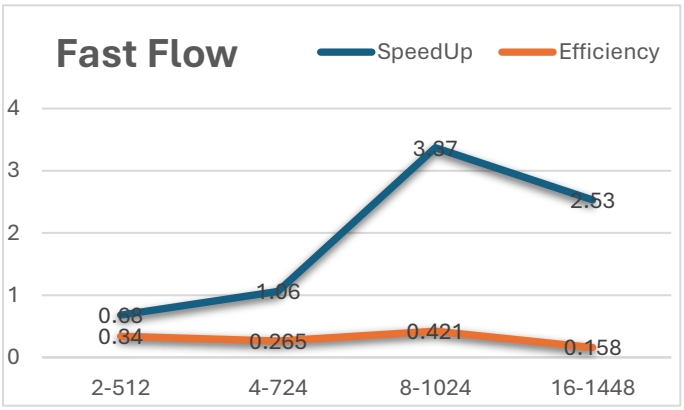


In the 1024 matrix, strong scalability is very limited, and efficiency is low starting from just 4 processors, with speed-up close to zero. This demonstrates how MPI is not well-suited for such small problems.

In contrast, for the 4096 matrix, a decent speed-up is observed for larger problems, but efficiency remains low compared to FastFlow, suggesting that my MPI version with this sizes of problems suffers from comunication overhead.

- Weak Scaling :





In this case, by increasing the number of processors, we ideally want the efficiency to remain constant. In the FastFlow version, this was more or less achieved.

With the increase in the number of threads and matrix size, efficiency slightly decreases but remains acceptable. This indicates that FastFlow scales reasonably well in terms of weak scalability, effectively handling a larger workload with more threads.

MPI shows weak weak scalability. Efficiency remains low even as the number of processors increases along with the matrix size. This implies that the communication overhead between processors remains a limiting factor.